COMP108
Data Structures and Algorithms

Greedy Algorithm (Part II Minimum Spanning Tree)

Professor Prudence Wong

pwong@liverpool.ac.uk

2025-26

Minimum Spanning Tree (MST)

Given an undirected connected graph G

- The edges are labelled by weight

Spanning tree of G

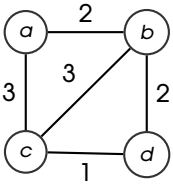
- a tree containing all vertices in G

Minimum spanning tree of G

- a spanning tree of G with minimum weight

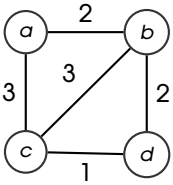
Examples

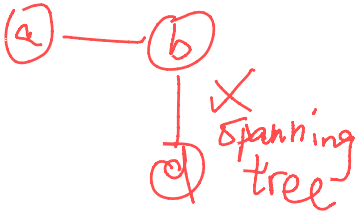
Graph G (edge label is weight)



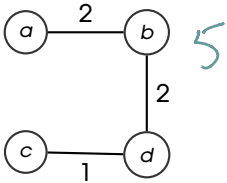
Examples

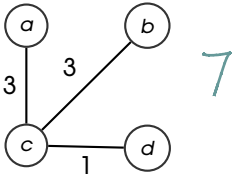
Graph G (edge label is weight)

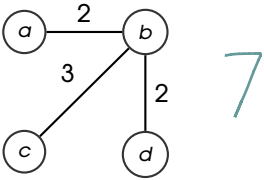




Spanning trees of G

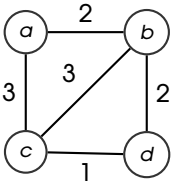




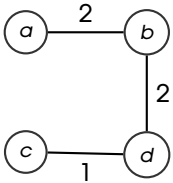


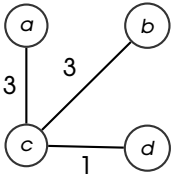
Examples

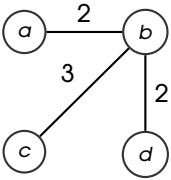
Graph G (edge label is weight)



Spanning trees of G



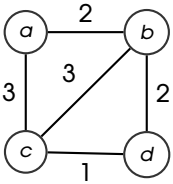




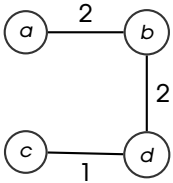
Which is MST?

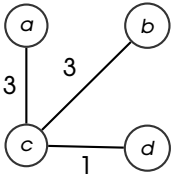
Examples

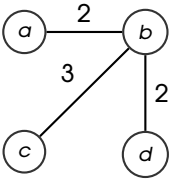
Graph G (edge label is weight)



Spanning trees of G



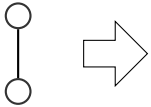




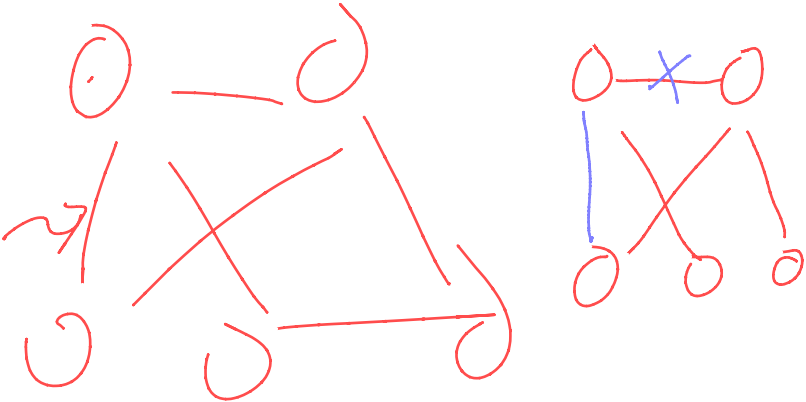
Which is MST?

How many possible spanning trees?

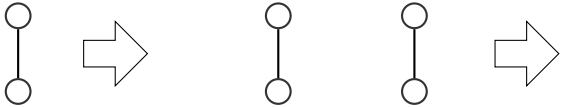
Idea of Kruskal's algorithm - MST



min-weight edge



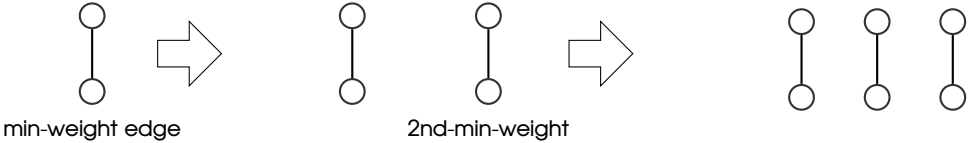
Idea of Kruskal's algorithm - MST



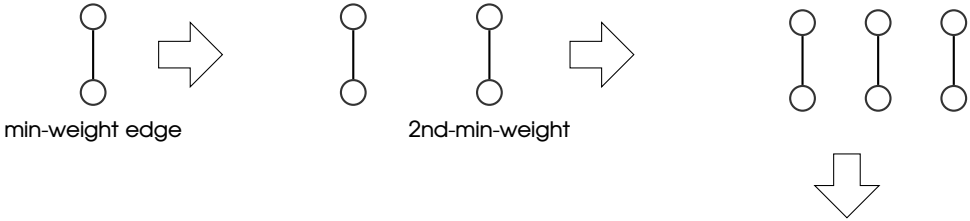
min-weight edge

2nd-min-weight

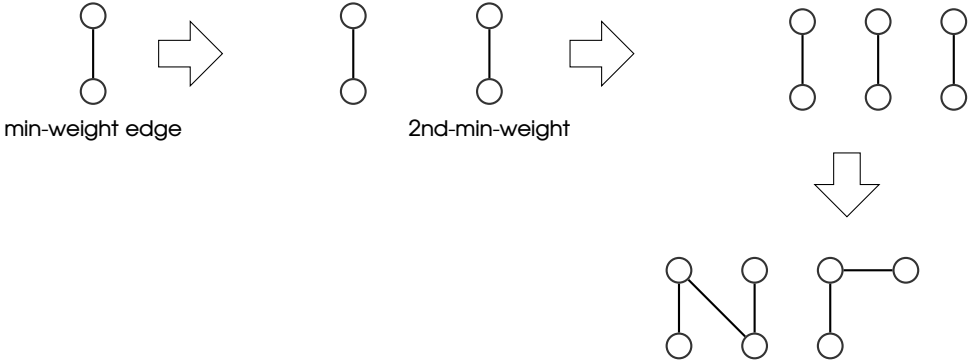
Idea of Kruskal's algorithm - MST



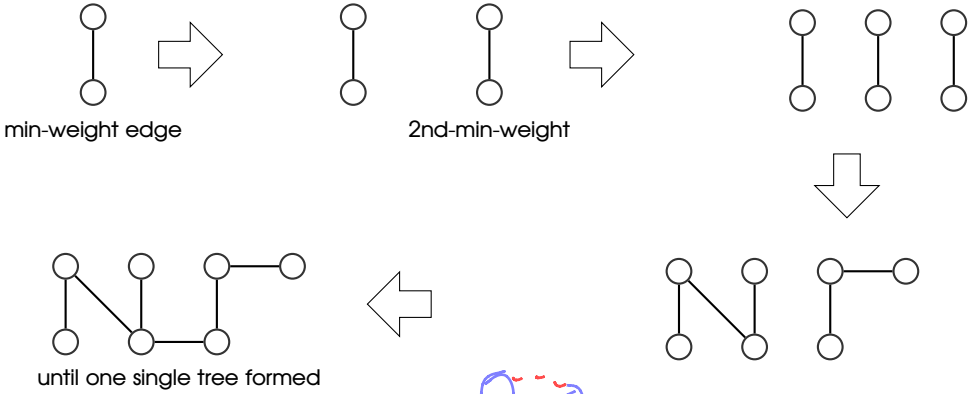
Idea of Kruskal's algorithm - MST



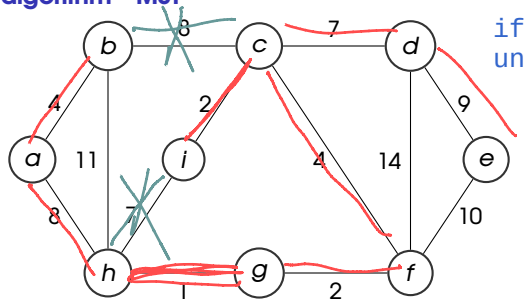
Idea of Kruskal's algorithm - MST



Idea of Kruskal's algorithm - MST



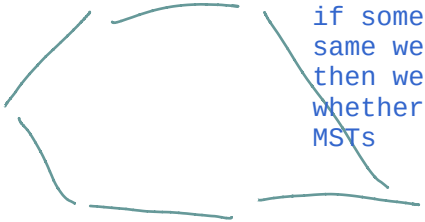
Kruskal's algorithm - MST



if all edges have different weight
unique MST

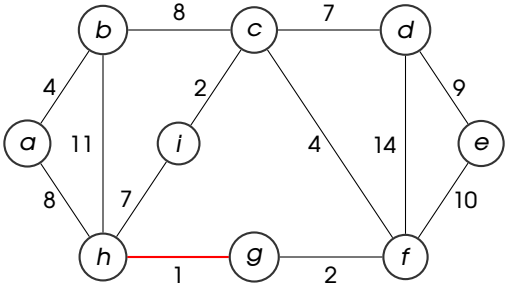
✓	(g, h)	1
✓	(c, i)	2
✓	(f, g)	2
✓	(a, b)	4
✓	(c, f)	4
✓	(c, d)	7
✗	(h, i)	7
✓	(a, h)	8
✗	(b, c)	8
✓	(d, e)	9
	(e, f)	10
	(b, h)	11
	(d, f)	14

Arrange edges from smallest to largest weight



if some edges have
same weight
then we don't know
whether there are multiple
MSTs

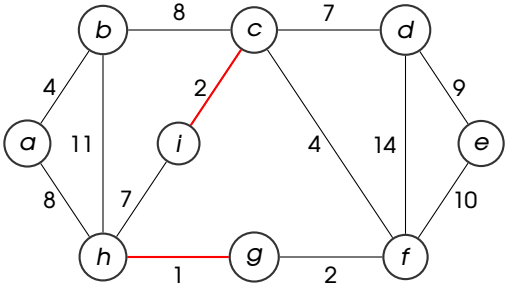
Kruskal's algorithm - MST



Choose the minimum weight edge

✓	(g, h)	1
	(c, i)	2
	(f, g)	2
	(a, b)	4
	(c, f)	4
	(c, d)	7
	(h, i)	7
	(a, h)	8
	(b, c)	8
	(d, e)	9
	(e, f)	10
	(b, h)	11
	(d, f)	14

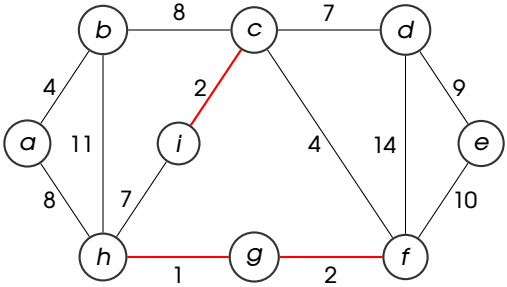
Kruskal's algorithm - MST



Choose the next minimum weight edge

✓	(g, h)	1
✓	(c, i)	2
	(f, g)	2
	(a, b)	4
	(c, f)	4
	(c, d)	7
	(h, i)	7
	(a, h)	8
	(b, c)	8
	(d, e)	9
	(e, f)	10
	(b, h)	11
	(d, f)	14

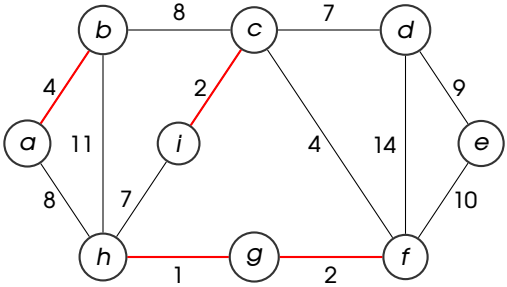
Kruskal's algorithm - MST



✓	(g, h)	1
✓	(c, i)	2
✓	(f, g)	2
	(a, b)	4
	(c, f)	4
	(c, d)	7
	(h, i)	7
	(a, h)	8
	(b, c)	8
	(d, e)	9
	(e, f)	10
	(b, h)	11
	(d, f)	14

Continue as long as no cycle forms

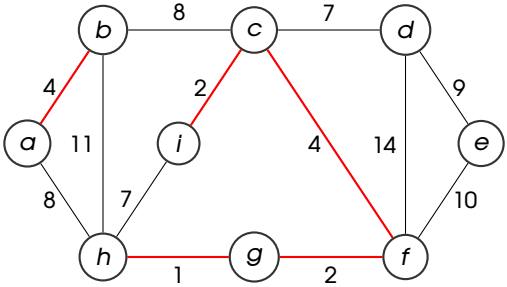
Kruskal's algorithm - MST



✓	(g, h)	1
✓	(c, i)	2
✓	(f, g)	2
✓	(a, b)	4
	(c, f)	4
	(c, d)	7
	(h, i)	7
	(a, h)	8
	(b, c)	8
	(d, e)	9
	(e, f)	10
	(b, h)	11
	(d, f)	14

Continue as long as no cycle forms

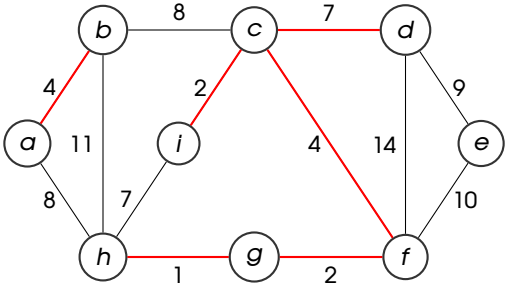
Kruskal's algorithm - MST



✓	(g, h)	1
✓	(c, i)	2
✓	(f, g)	2
✓	(a, b)	4
✓	(c, f)	4
	(c, d)	7
	(h, i)	7
	(a, h)	8
	(b, c)	8
	(d, e)	9
	(e, f)	10
	(b, h)	11
	(d, f)	14

Continue as long as no cycle forms

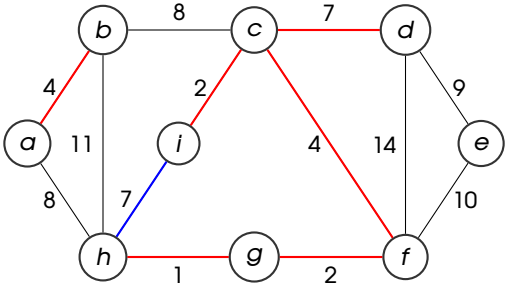
Kruskal's algorithm - MST



✓	(g, h)	1
✓	(c, i)	2
✓	(f, g)	2
✓	(a, b)	4
✓	(c, f)	4
✓	(c, d)	7
	(h, i)	7
	(a, h)	8
	(b, c)	8
	(d, e)	9
	(e, f)	10
	(b, h)	11
	(d, f)	14

Continue as long as no cycle forms

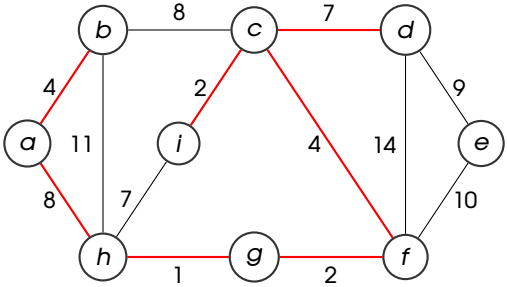
Kruskal's algorithm - MST



✓	(g, h)	1
✓	(c, i)	2
✓	(f, g)	2
✓	(a, b)	4
✓	(c, f)	4
✓	(c, d)	7
	(h, i)	7
	(a, h)	8
	(b, c)	8
	(d, e)	9
	(e, f)	10
	(b, h)	11
	(d, f)	14

next edge cannot be included, otherwise, cycle is formed

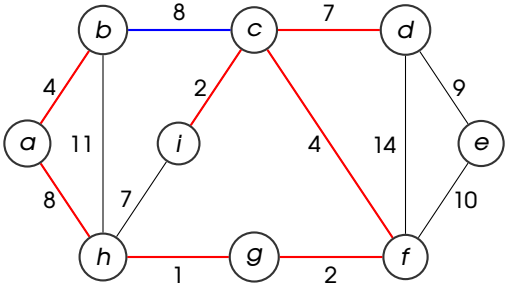
Kruskal's algorithm - MST



✓	(g, h)	1
✓	(c, i)	2
✓	(f, g)	2
✓	(a, b)	4
✓	(c, f)	4
✓	(c, d)	7
	(h, i)	7
✓	(a, h)	8
	(b, c)	8
	(d, e)	9
	(e, f)	10
	(b, h)	11
	(d, f)	14

Choose the next minimum weight edge

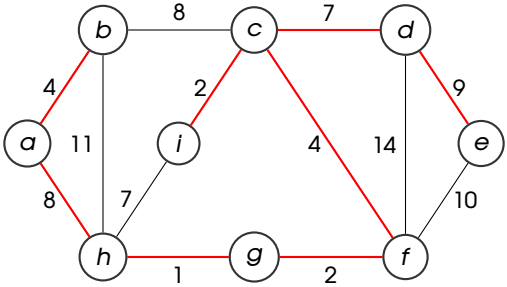
Kruskal's algorithm - MST



✓	(g, h)	1
✓	(c, i)	2
✓	(f, g)	2
✓	(a, b)	4
✓	(c, f)	4
✓	(c, d)	7
	(h, i)	7
✓	(a, h)	8
	(b, c)	8
	(d, e)	9
	(e, f)	10
	(b, h)	11
	(d, f)	14

next edge cannot be included, otherwise, cycle is formed

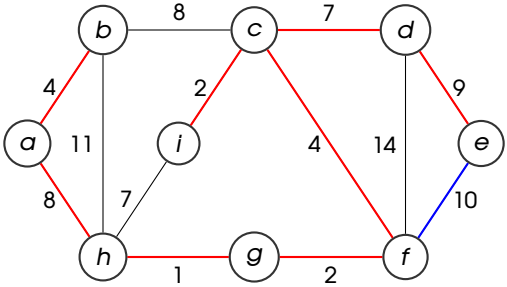
Kruskal's algorithm - MST



Choose the next minimum weight edge

✓	(g, h)	1
✓	(c, i)	2
✓	(f, g)	2
✓	(a, b)	4
✓	(c, f)	4
✓	(c, d)	7
	(h, i)	7
✓	(a, h)	8
	(b, c)	8
✓	(d, e)	9
	(e, f)	10
	(b, h)	11
	(d, f)	14

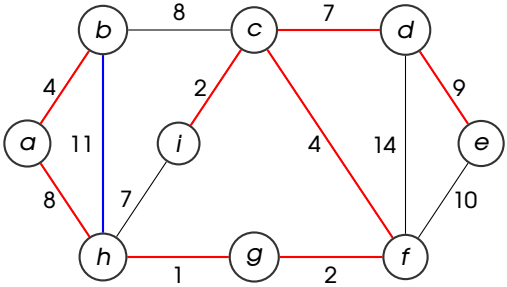
Kruskal's algorithm - MST



✓	(g, h)	1
✓	(c, i)	2
✓	(f, g)	2
✓	(a, b)	4
✓	(c, f)	4
✓	(c, d)	7
	(h, i)	7
✓	(a, h)	8
	(b, c)	8
✓	(d, e)	9
	(e, f)	10
	(b, h)	11
	(d, f)	14

next edge cannot be included, otherwise, cycle is formed

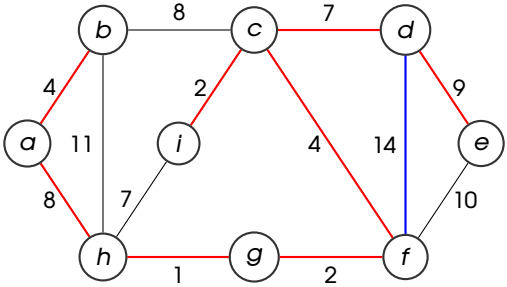
Kruskal's algorithm - MST



✓	(g, h)	1
✓	(c, i)	2
✓	(f, g)	2
✓	(a, b)	4
✓	(c, f)	4
✓	(c, d)	7
	(h, i)	7
✓	(a, h)	8
	(b, c)	8
✓	(d, e)	9
	(e, f)	10
	(b, h)	11
	(d, f)	14

next edge cannot be included, otherwise, cycle is formed

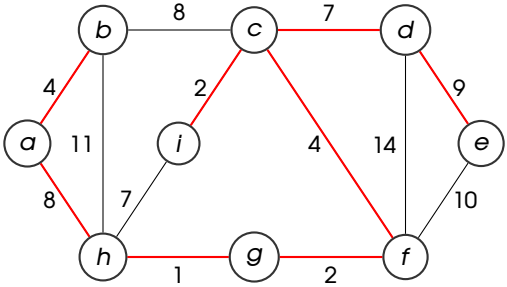
Kruskal's algorithm - MST



✓	(g, h)	1
✓	(c, i)	2
✓	(f, g)	2
✓	(a, b)	4
✓	(c, f)	4
✓	(c, d)	7
	(h, i)	7
✓	(a, h)	8
	(b, c)	8
✓	(d, e)	9
	(e, f)	10
	(b, h)	11
	(d, f)	14

next edge cannot be included, otherwise, cycle is formed

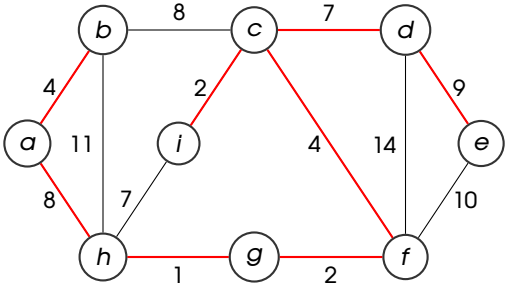
Kruskal's algorithm - MST



✓	(g, h)	1
✓	(c, i)	2
✓	(f, g)	2
✓	(a, b)	4
✓	(c, f)	4
✓	(c, d)	7
	(h, i)	7
✓	(a, h)	8
	(b, c)	8
✓	(d, e)	9
	(e, f)	10
	(b, h)	11
	(d, f)	14

MST is found when all edges are examined

Kruskal's algorithm - MST



✓	(g, h)	1
✓	(c, i)	2
✓	(f, g)	2
✓	(a, b)	4
✓	(c, f)	4
✓	(c, d)	7
	(h, i)	7
✓	(a, h)	8
	(b, c)	8
✓	(d, e)	9
	(e, f)	10
	(b, h)	11
	(d, f)	14

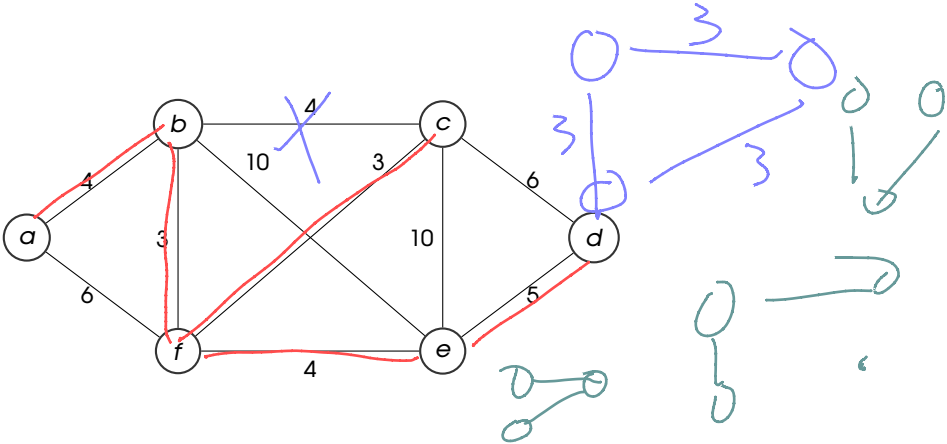
Order of edges selected (g, h), (c, i), (f, g), (a, b), (c, f), (c, d), (a, h), (d, e)

Kruskal's algorithm - MST

Kruskal's algorithm is **greedy** in the sense that

- it always attempt to select the **smallest** weight edge to be included in the MST

Exercise - Find MST for this graph



Order of edges selection:

```
If there are multiple edges having the same weig
ht, then there must be multiple MSTs
(FALSE)
```

Pseudo code

// Given an undirected connected graph $G = (V, E)$

$T \leftarrow \emptyset$

$E' \leftarrow E$

sort E' from small to large weight

while $E' \neq \emptyset$ do

begin

end

Pseudo code

// Given an undirected connected graph $G = (V, E)$

$T \leftarrow \emptyset$

$E' \leftarrow E$

sort E' from small to large weight

while $E' \neq \emptyset$ do

begin

pick an edge ***e*** in E' with minimum weight

end

Pseudo code

// Given an undirected connected graph $G = (V, E)$

$T \leftarrow \emptyset$
$E' \leftarrow E$
sort E' from small to large weight
while $E' \neq \emptyset$ do
begin
    pick an edge $\boldsymbol{e}$ in E' with minimum weight
    if adding e to T does **not** form cycle then
        add e to T, i.e., $\boldsymbol{T} \leftarrow \boldsymbol{T} \cup \{\boldsymbol{e}\}$

end

Pseudo code

// Given an undirected connected graph $G = (V, E)$

$T \leftarrow \emptyset$
$E' \leftarrow E$
sort E' from small to large weight
while $E' \neq \emptyset$ do
begin
    pick an edge $\boldsymbol{e}$ in E' with minimum weight
    if adding e to T does **not** form cycle then
        add e to T, i.e., $\boldsymbol{T} \leftarrow \boldsymbol{T} \cup \{\boldsymbol{e}\}$
    remove e from E', i.e., $\boldsymbol{E'} \leftarrow \boldsymbol{E'} \setminus \{\boldsymbol{e}\}$
end

Pseudo code

// Given an undirected connected graph $G = (V, E)$

$T \leftarrow \emptyset$
$E' \leftarrow E$
sort E' from small to large weight
while $E' \neq \emptyset$ do
begin
  pick an edge $\boldsymbol{e}$ in E' with minimum weight
  if adding e to T does **not** form cycle then
    add e to T, i.e., $\boldsymbol{T} \leftarrow \boldsymbol{T} \cup \{\boldsymbol{e}\}$
  remove e from E', i.e., $\boldsymbol{E'} \leftarrow \boldsymbol{E'} \setminus \{\boldsymbol{e}\}$
end

How to determine if cycle is formed?

How to determine if cycle is formed?

Use coloring - initially all vertices are un-colored

When we consider a new edge $e = (u, v)$, there may be three cases:

1. at most one of u and v is end point of an already chosen edge

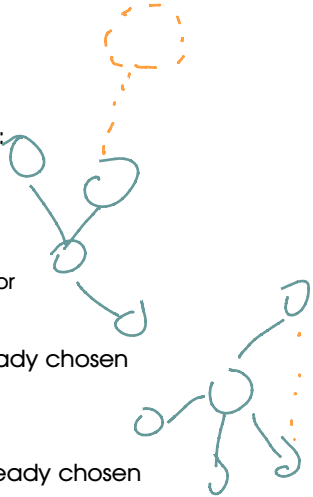
2. u and v both belong to the same partial tree that has been already chosen

3. u and v belong to two separate partial trees that have been already chosen

How to determine if cycle is formed?

Use coloring - initially all vertices are un-colored

When we consider a new edge $e = (u, v)$, there may be three cases:



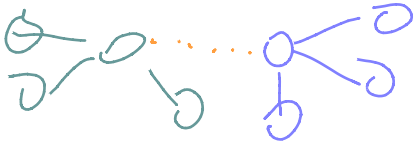
1. at most one of u and v is end point of an already chosen edge
   - if both u & v uncolored, color both u & v with a new color
   - if only one colored, color the uncolored vertex with the same color
2. u and v both belong to the same partial tree that has been already chosen
3. u and v belong to two separate partial trees that have been already chosen

How to determine if cycle is formed?

Use coloring - initially all vertices are un-colored

When we consider a new edge $e = (u, v)$, there may be three cases:

1. at most one of u and v is end point of an already chosen edge
   - if both u & v uncolored, color both u & v with a new color
   - if only one colored, color the uncolored vertex with the same color
2. u and v both belong to the same partial tree that has been already chosen
   - if both u & v are colored the same color, then should NOT select e
3. u and v belong to two separate partial trees that have been already chosen



How to determine if cycle is formed?

Use coloring - initially all vertices are un-colored

When we consider a new edge $e = (u, v)$, there may be three cases:

1. at most one of u and v is end point of an already chosen edge
   - if both u & v uncolored, color both u & v with a new color
   - if only one colored, color the uncolored vertex with the same color
2. u and v both belong to the same partial tree that has been already chosen
   - if both u & v are colored the same color, then should NOT select e
3. u and v belong to two separate partial trees that have been already chosen
   - if u & v with different color, choose one color $color(u)$ and recolor ALL vertices with $color(v)$ to $color(u)$

Pseudo code

// Given an undirected connected graph $G = (V, E)$

$T \leftarrow \emptyset$

$E' \leftarrow E$

sort E' from small to large weight

while $E' \neq \emptyset$ do

begin

pick an edge e in E' with minimum weight

if adding e to T does **not** form cycle then

add e to T, i.e., $T \leftarrow T \cup \{e\}$

remove e from E', i.e., $E' \leftarrow E' \setminus \{e\}$

end

$O(m^2)$

$O(m \log m)$

$O(m)$

$O(n)$

nC_2

C_2^n

```
n = |V|, m = |E|
O(mn + m log m) = O(mn)
```

Time complexity?

```
what is max value of m?
 O(n^2)

log n^2 = 2 log n
```

Summary

Summary: Kruskal's algorithm for Minimum Spanning Tree

Next: Dijkstra's algorithm for Single-Source Shortest-Paths

For note taking